Вычисления и память

Повторение

- Какие виды поведения бывают?

- Чем UB отличается от IDB и Unspecified?

Повторение

- Какие виды поведения бывают?
  - Specified – поведение специфицировано и задокументировано
  - Unspecified – поведение неспецифицировано и не задокументировано
  - Implementation defined – поведение неспецифицировано, но задокументировано
  - Undefined – поведение неопределено.

- Чем UB отличается от IDB и Unspecified?
  - UB – ошибочное поведение, может случится вообще что угодно

  - IDB – поведение корректно, но зависит от конкретного компилятора. Что именно будет – читать в документации конкретного компилятора.

  - Unspec – поведение корректно, но что именно произойдет мы точно не знаем. Но ничего плохого не случится.

Два разных инкремента

Оператор инкремента ++ увеличивает текущее значение переменной на 1.

Но чем отличаются два оператора?

```
1 int i = 0;
2
3 i++;
4
5 ++i;
```

Два разных инкремента

Оператор инкремента ++ увеличивает текущее значение переменной на 1.

Но чем отличаются два оператора?

Выше – постфиксный. Увеличивает на 1.
Возвращает старое значение

Ниже – префиксный. Увеличивает на 1.
Возвращает новое значение

```
1 int i = 0;
2
3 i++;
4
5 ++i;
```

Два разных инкремента

Что выведет программа справа?

```
1 int i = 0;
2 int x = 0;
3
4 x = i++;
5 printf("%d\n", x);
6
7 x = ++i;
8 printf("%d\n", x);
```

Побочные эффекты и результат выражения

Побочный эффект – наблюдаемые изменения от выполнения операции (изменения переменных, ввод и вывод, запись в память и т.д.)

Выражение само по себе вычисляет значение. Иногда выражения имеют побочные эффекты.

Что в программе справа является побочным эффектом?

```
1 int i = 0;
2 int x = 0;
3
4 x = i++;
5 printf("%d\n", x);
6
7 x = ++i;
8 printf("%d\n", x);
```

Побочные эффекты

Определите тип результата и побочные эффекты для выражений ниже

```
int i = 42, a = 100;
```

Выражение	Значение
`i++ + 15L`	57
`++i + 015`	56
`a = i += i`	84
`a > 0 ? ++i : i++`	43

Побочные эффекты

Определите тип результата и побочные эффекты для выражений ниже

```
int i = 42, a = 100;
```

Выражение	Значение	Тип	Побочный эффект
`i++ + 15L`	57	`long`	`i = 43`
`++i + 015`	56	`int`	`i = 43`
`a = i += i`	84	`int`	`i = 84, a = 84`
`a > 0 ? ++i : i++`	43	`int`	`i = 43`

Точки следования

Вычисление побочных эффектов происходит в любой момент времени.

Компилятор вправе вставить вычисление побочных эффектов в любое место в программе до **точки следования**

Точка следования – место в программе, где все предыдущие побочные эффекты уже завершились, а новые еще не начались

Точки следования

Точки следования стоят:

В конце полного выражения (точка с запятой)

В логических выражения – после вычисления левого операнда (ленивые вычисления)

При операторе запятая – после вычисления левого операнда

При вызовах функции – перед вызовом аргументы вычислены

В тернарном операторе – после вычисления условия

```
 1 i = 10 + x - y;
 2
 3 a && b
 4 a || b
 5
 6 a, b
 7
 8 f(a + 1, b++);
 9
10 c ? a : b;
```

Точки следования и UB

Между двумя точками следования переменную нельзя

1. изменить дважды
2. изменить и прочитать в другом месте выражения

Исключение: i = i + 1;

Нарушение – UB

```
1 int i = 0;
2
3 int x = i++ + i++;
4 int y = (i = 1) + i++;
```

to UB or not to UB

```
i = i + 1;                          // ?
i = i++;                            // ?
j = i++ + i++;                      // ?
j = i++ && i++;                     // ?
j = (i++, i++);                     // ?
printf("%d %d\n", i, i++);          // ?
```

to UB or not to UB

```
i = i + 1;                          // ok
i = i++;                            // UB
j = i++ + i++;                      // UB
j = i++ && i++;                     // ok
j = (i++, i++);                     // ok
printf("%d %d\n", i, i++);          // UB
```

Основные операции языка

1. Условные операторы
   a. if - else
   b. swtich
   c. тернарный оператор

2. Циклы:
   a. while
   b. for
   c. do-while

Условные операторы

```
1 if (condition) {
2     // code
3 }
4
5 if (condition) {
6   // some code
7 } else {
8   // some code
9 }
```

```
 1 switch (condition) {
 2   case 0:
 3     // code
 4     break;
 5   case 128:
 6     // code
 7     break;
 8
 9   default:
10     // code
11     break;
12 }
```

Условные операторы

```
1 x = conition ? if_true : if_false;
2
3 if (condition)
4   x = if_true;
5 else
6   x = if_false;
```

Циклы

```
1 for (int i = 0; i < size; i = i + 10) {
2   // code
3 }
```

```
1 int i = 0;
2 // ...
3 for (; i < size; i++) {
4   // code
5 }
```

Циклы

```
1 while (condition) {
2   // code
3 }
4
```

Циклы

```
1 do {
2   // code executes at least once
3 } while (condition);
4
```

Циклы. break и continue

Иногда нужно завершить цикл досрочно.
Для этого используют ключевое слово break.

Иногда нужно пропустить итерацию.
Для этого используют ключевое слово continue.

```
while (condition) {
  try_do_work();

  if (work_done)
    break;

  if (!need_extra)
    continue

  extra();
}
```

Указатели

Указатель – переменная специального типа, которая хранит адрес в памяти другой переменной.

Операции:

Разыменование * – получение доступа к данным, на которые указываем

Взятие адреса & – получить адрес данных.

Указатели

```
1 int x = 0;
2
3 // get address of x
4 int* px = &x;
5
6 x = 10;
7
8 // get data that is pointed by px
9 int y = *px + 1;
```

Указатели

Указатель – переменная специального типа, которая хранит адрес в памяти другой переменной.

Указатель всегда бывает на “что-то”:

- int* pi – указатель на int. pi хранит адрес переменной типа int
- float* pf – указатель на float. pf хранит адрес переменной типа float
- void (*pfunc) (int) – указатель функцию типа void (int). pfunc хранит адрес места в памяти где лежит код функции void (int)

Указатели

Указатель – переменная специального типа, которая хранит адрес в памяти другой переменной.

“Особый” указатель – void*

Указатель “ни на что”. Используется для абстракций. Напрямую такой указатель нельзя разыменовывать и складывать.

Арифметика указателей

При сложении указателей, домножение на размер типа происходит автоматически.

Размер типа берется из типа указателя.

```
1 int x = 7;
2 int *px = &x;
3 char *pc = (char*)&x;
4
5 px + 1; // -> px + sizeof(int)  = px + 4
6 pc + 1; // -> pc + sizeof(char) = pc + 1
7
```

Массивы

Массив – это структура *однотипных* данных, занимающих *непрерывную* область памяти

Главная особенность массива в том, что данные лежат непрерывно

Зная адрес начала и размер элемента, мы можем получить доступ к i-му элементу:

```
a[i] = *(base_address + i*sizeof(a[0]))
```

Массивы и указатели

Имя массива неявно приводится к указателю на его начало. Это называется array to pointer decay.

С помощью этого можно передавать массивы в функции как обычные указатели.

Рекап

- Точки следования разделяют сайд-эффекты в программе
- Нужно быть осторожным, чтоб не выстрелить себе в ноги
- Нужно быть осторожным с {}, чтоб не поставить ; после if / while / for
- Указатели – специальный тип, который хранит адрес другой переменной
- Арифметика указателей уже учитывает тип указателя
- Массив – набор подряд лежащих однотипных данных